

【pytorch03】梯度下降

2026年9月7日 11:18

- **优化问题**: $w^* = \operatorname{argmin} \operatorname{cost}(w)$ (找到使代价最小的权重)。

- **梯度下降算法 (GD)** : (贪心: 每一步最优)

1. 随机初始化权重;
2. 计算代价函数关于 w 的梯度 (导数) ;
3. 更新权重: $w = w - \alpha \cdot \partial \operatorname{cost} / \partial w$; α 表示学习率
4. 循环迭代直到收敛。

- *问题: 只能找到局部最优, 而非全局最优*

- *鞍点: 梯度为0→会导致无法迭代*

- **学习率 α (Learning Rate)** : 控制每次更新步长, 是超参数。

- **代价函数 (MSE) 对 w 的梯度:**

$$\partial \operatorname{cost} / \partial w = (1/N) \cdot \sum 2 \cdot x_i \cdot (x_i \cdot w - y_i)。$$

- **推导过程:**

$$\begin{aligned} \frac{\partial \operatorname{cost}(\omega)}{\partial \omega} &= \frac{\partial}{\partial \omega} \frac{1}{N} \sum_{n=1}^N (x_n \cdot \omega - y_n)^2 \\ &= \frac{1}{N} \sum_{n=1}^N \frac{\partial}{\partial \omega} (x_n \cdot \omega - y_n)^2 \\ &= \frac{1}{N} \sum_{n=1}^N 2 \cdot (x_n \cdot \omega - y_n) \frac{\partial (x_n \cdot \omega - y_n)}{\partial \omega} \\ &= \frac{1}{N} \sum_{n=1}^N 2 \cdot x_n \cdot (x_n \cdot \omega - y_n) \end{aligned}$$

- **实现要素**: forward、cost、gradient 三个函数 + 训练循环 (epoch) 。

- **随机梯度下降 (SGD)** :

- GD 用「所有样本的代价 **cost**」的梯度更新一次;
 - SGD 用「单个样本的损失 **loss**」的梯度逐个更新 (每样本更新一次) 。
 - $w = w - \alpha \cdot \partial \operatorname{loss} / \partial w$

- $$\frac{\partial \operatorname{loss}_n}{\partial \omega} = 2 \cdot x_n \cdot (x_n \cdot \omega - y_n)$$

- 有可能跨越鞍点

- **本质区别**: GD 的梯度是全体平均、更新慢但稳; SGD 逐样本更新、更快但更抖。

- **解决方法**: **mini-batch** 用一小批量进行随机梯度下降